Práctica Preparcial

Sistemas Digitales

Segundo Cuatrimestre 2026

Ejercicio 1 Un dispositivo recibe palabras de 6 bits. Se consideran:
A = 011101
B = 000111
D = 111101

a) Interpretar A, B, D en sin signo y en complemento a 2 (CA2).

b) Reescribir cada patrón en sistema hexadecimal, respetando el ancho original de 6 bits. ¿Qué pasa si los extiendo a 8 bits?

c) Calcular A+B, B+D usando un sumador de 6 bits. Para cada cuenta, dar los flags CVZN y determinar si el resultado obtenido representa la suma matemática.

d) Volver a calcular ambas sumas usando un sumador de 8 bits, extendiendo de forma apropiada para cada sistema de representación. Explicar qué problemas de representabilidad desaparecen.

e) Observar los resultados del inciso e). ¿Un acarreo implica un resultado incorrecto en CA2? ¿Por qué?

Ejercicio 2 Se desea construir un circuito que dados dos bits (E_1,E_0) que codifica el número E en notación complemento, genere otros dos bits (S_1, S_0) que codifica el número $S = E-1$ en la misma notación, y un tercer bit V que indica la condición *no representable* ($V = 1 \iff E-1$ no es representable con 2 bits en notación complemento; en este caso particular, los valores de S_1 y S_0 serán irrelevantes)

a) Escriba las tablas de verdad para S_1, S_0 y V en función de los valores de E_1 y E_0.

b) Escriba las ecuaciones lógicas para las tres salidas en función de las dos entradas

c) Implemente el circuito en SystemVerilog usando, a lo sumo, una compuerta NOT, una OR y una AND.

d) Dibujar el circuito similar a la sintetización en HDL según la implementación previa.

Ejercicio 3 Un caracol robótico avanza de izquierda a derecha sobre una cinta que contiene una secuencia de bits. En cada flanco ascendente de *clock*, lee el bit que está debajo de su antena y avanza al siguiente:

- La entrada A es el bit de la cinta que está leyendo.
- La salida Y vale 1 cuando el caracol sonríe.
- El caracol **sonríe si los últimos dos bits leídos fueron 01**. En los demás casos, Y=0.
- Al hacer reset, el caracol todavía no leyó ningún bit y Y=0.

a) Diseñar el diagrama de estados de una FSM de Moore que implemente este comportamiento. Marcar el estado inicial, las condiciones en cada transición y la salida en cada estado.

b) Codificar los estados de Moore con la menor cantidad de bits posible. Construir la tabla de próximo estado y la tabla de salidas. Indicar qué sucede con las codificaciones que no se utilizan.

c) A partir de las tablas del ejercicio 2, obtener y simplificar las ecuaciones booleanas de los bits de próximo estado ‘n1‘, ‘n0‘ y de la salida ‘Y‘ en función del estado actual ‘s1s0‘ y de la entrada ‘A‘. Explicitar cómo se trata el estado no utilizado.

d) Diseñar una FSM de Mealy para el mismo problema, dibujar su diagrama y escribir su tabla de transición/salida. Comparar la cantidad de estados y cuándo puede cambiar la salida. Usar la entrada ‘0100110111‘ como ejemplo para identificar cuándo sonríe.

Ejercicio 4 na máquina tiene un acumulador ACC de 8 bits, un registro temporal T de 8 bits y una ALU combinacional. La ALU puede pasar su segundo operando, sumar, restar o hacer AND. Los resultados aritméticos conservan sus 8 bits bajos.

Las instrucciones tienen 6 bits:

00 IIII

IIII es un inmediato de 4 bits en CA2, que se extiende con signo a 8 bits, incluso para AND.

00	Instrucción	Efecto
00	LOAD inmediato	ACC = inmediato extendido
01	ADD inmediato	ACC = ACC + inmediato extendido
10	SUB inmediato	ACC = ACC - inmediato extendido
11	AND inmediato	ACC = ACC AND inmediato extendido

La instrucción ya está en IR y permanece estable durante su ejecución. Cada instrucción usa dos estados, uno por ciclo:

- CALCULAR: al terminar el ciclo, T captura el resultado de la ALU.
- GUARDAR: al terminar el ciclo, ACC captura T.

Al reset, ACC=T=0 y el controlador queda en CALCULAR. Un mecanismo externo, que no se debe diseñar, presenta la primera instrucción antes de CALCULAR y actualiza IR después de cada flanco de GUARDAR, a tiempo para el siguiente ciclo. No contar ciclos de búsqueda ni carga de IR.

a) Dibujar el camino de datos, incluyendo extensión de signo, ALU y conexiones entre registros. Identificar señales de datos y de control.

b) Completar la tabla de control con operación de ALU, IdT y IdACC. Indicar cómo se obtienen los campos desde IR y cómo transita el controlador.

c) Codificar y ejecutar el siguiente programa. Mostrar ACC y T después de cada flanco e indicar el número total de ciclos.

LOAD -3
ADD 7
SUB -2

d) Explicar qué sucede si, con el camino original, se activan IdT y IdACC en el primer flanco, intentando ejecutar una instrucción completa de una vez. Mostrar el caso de LOAD -3 desde el reset.